

Robot Soccer Self-Localization with Active Inference

Robot football?

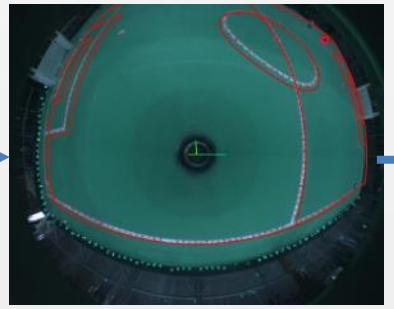


As-is algorithm to determine the robot field position

Omi-camera



Line detects



- Line detects based on white-green edges in the camera image
- reference points (100) on the detected lines (red crosses)

in camera coordinate system (CCS)

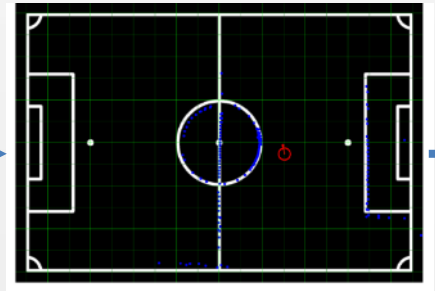
Detected lines relative to robot



- Dewarp points to coordinates relative to the robot

In robot coordinate system (RCS)

Search for possible field positions



- Algorithm for searching the maximum fitness for x, y, θ on a reference field
- 25 hypotheses by 3 different algorithms

In field coordinate system (FCS)

Most likely field position



- Kalman filter on previous positions, odometry and the 25 hypotheses (and some magic) to select the most probable location

In field coordinate system (FCS)

As-is algorithm to determine the robot field position

Current MSL wheeled platform

Algorithm is:

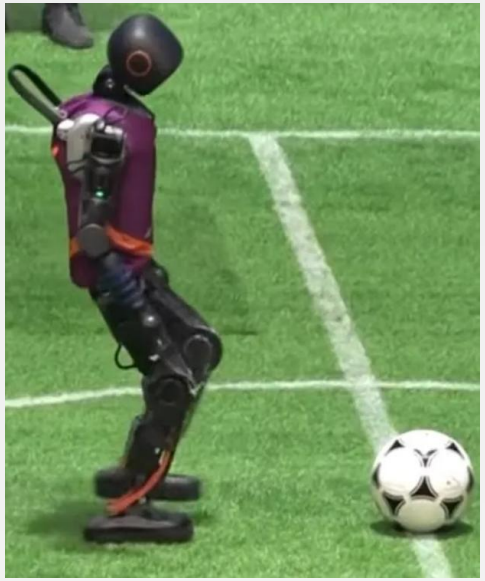
- error prone
  - robots run sometimes out of the field or get “lost” (jumps in position)
- Very sensitive to lighting conditions
  - Need to recalibrate under different lighting conditions (e.g. sunshine through windows)



Nextgen humanoid robots

Algorithm is not future proof

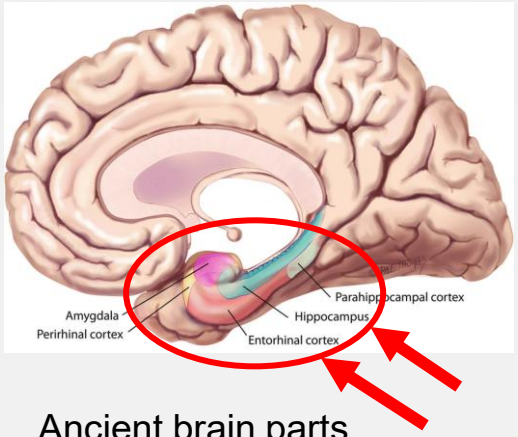
- No 360 degrees omni-camera
  - No complete field images to see all lines



While humans somehow know where they are



Brain components of navigation



Ancient brain parts, early in evolution

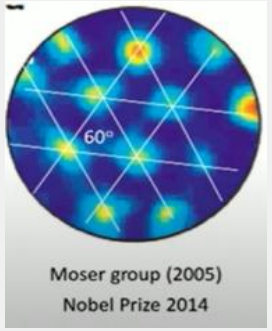
Head direction cells

- Provide a consistent sense of **direction**/orientation
- medial entorhinal cortex contains velocity encoding neurons



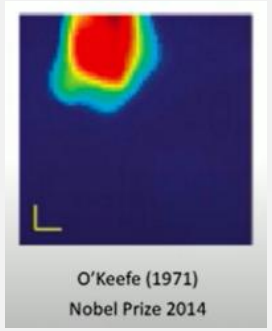
Grid cells

- Neural representation of space
- Tile the environment in a 2-dimensional honeycomb **map** fashion.
- In the medial entorhinal cortex



Place cells

- Become active only when a particular location (**landmark**) is observed
- In the hippocampus.



Navigation effectively exploits both:

Map based (Allocentric)



First person based (Egocentric)

- path-integration (dead reckoning)



Learning grid cells by predictive coding

Learning grid cells by predictive coding - Prof. Rafal Bogacz

- Grid cells are hidden states, that **predict** (and are learned from) place cell observations
- emerge robustly through predictive coding (active inference)


LEARNING GRID CELLS BY PREDICTIVE CODING

Mufeng Tang, Helen Barron & Rafal Bogacz
MRC Brain Network Dynamics Unit, University of Oxford
{mufeng.tang, helen.barron, rafal.bogacz}@bndu.ox.ac.uk

ABSTRACT

Grid cells in the medial entorhinal cortex (MEC) of the mammalian brain exhibit a strikingly regular hexagonal firing field over space. These cells are learned after birth and are thought to support spatial navigation but also more abstract computations. Although various computational models, including those based on artificial neural networks, have been proposed to explain the formation of grid cells, the process through which the MEC circuit learns to develop grid cells remains unclear. In this study, we argue that predictive coding, a biologically plausible plasticity rule known to learn visual representations, can also train neural networks to develop hexagonal grid representations from spatial inputs. We demonstrate that grid cells emerge robustly through predictive coding in both static and dynamic environments, and we develop an understanding of this grid cell learning capability by analytically comparing predictive coding with existing models. Our work therefore offers a novel and biologically plausible perspective on the learning mechanisms underlying grid cells. Moreover, it extends the predictive coding theory to the hippocampal formation, suggesting a unified learning algorithm for diverse cortical representations.

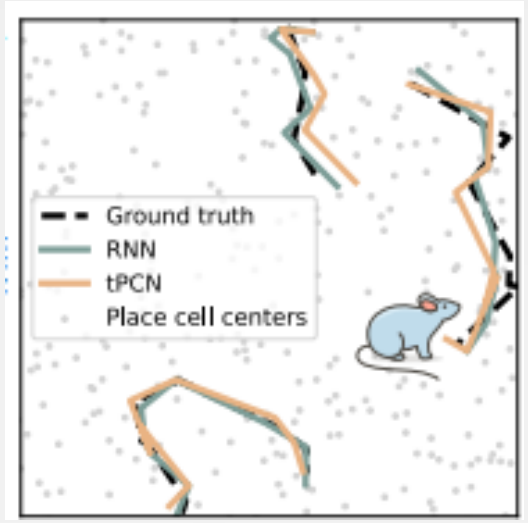
1 INTRODUCTION

Our brain contains a rich set of neural representations of space that help us navigate in an ever-changing world. These include hippocampal place cells (O'Keefe, 1976), which fire when an animal is at a specific spatial position, and grid cells observed in the medial entorhinal cortex (MEC) (Hafting et al., 2005), which fire when an animal occupies multiple positions on a hexagonal or triangular grid (Gil et al., 2018). Grid cells have been observed across various species (Yartsev et al., 2011; Jacobs et al., 2013; Killian et al., 2012; Ginosar et al., 2021; Gu et al., 2018), and play a crucial role in the computational mechanism underlying their emergence. Earlier models have focused on how mechanisms, such as membrane potential oscillation (O'Keefe & Burgess, 2005; Hasselmo et al., 2007) and specialized recurrent connectivity, can generate grid-like firing patterns (Fuhs & Touretzky, 2006; Burak & Fiete, 2009). More recently, research has shown that grid cells can emerge in recurrent neural networks (RNNs) trained using backpropagation through time (BPTT) for path integration tasks. The models are trained to predict their current location by integrating velocity inputs (Cueva & Wei, 2018; Banino et al., 2018; Whittington et al., 2020; Sorscher et al., 2023), providing a normative, task-driven account of the computational problem that the MEC grid cells address. However, the process by which the MEC circuit acquires, or learns the grid cells in a biologically plausible way has been largely neglected, despite the fact that grid cells are known to be learned, rather than hardwired at birth (Langston et al., 2010; Wills et al., 2010). Existing learning models (e.g. Weber & Sprekeler (2018)) are highly specialized for grid cells, and it is unclear whether plasticity rules for only one specific cell type exist in the brain.

In this paper, we directly tackle the learning problem underlying the emergence of grid cells using *predictive coding*, an algorithm modeling the plasticity rules for a variety of cortical functions and representations (Rao & Ballard, 1999; Friston, 2005). Our approach to modeling grid cell emergence through predictive coding is motivated by three key factors. Firstly, the predictive coding algorithm can be implemented in predictive coding networks (PCNs) with local computations and Hebbian plasticity (Bogacz, 2017), making it more biologically plausible than learning rules such as back-propagation. Secondly, PCNs have been successful in replicating representations in other regions of the brain, such as the visual cortex (Rao & Ballard, 1999; Olshausen & Field, 1996; Millidge et al.,

arXiv:2410.01022v2 [q-bio.NC] 4 Oct 2024

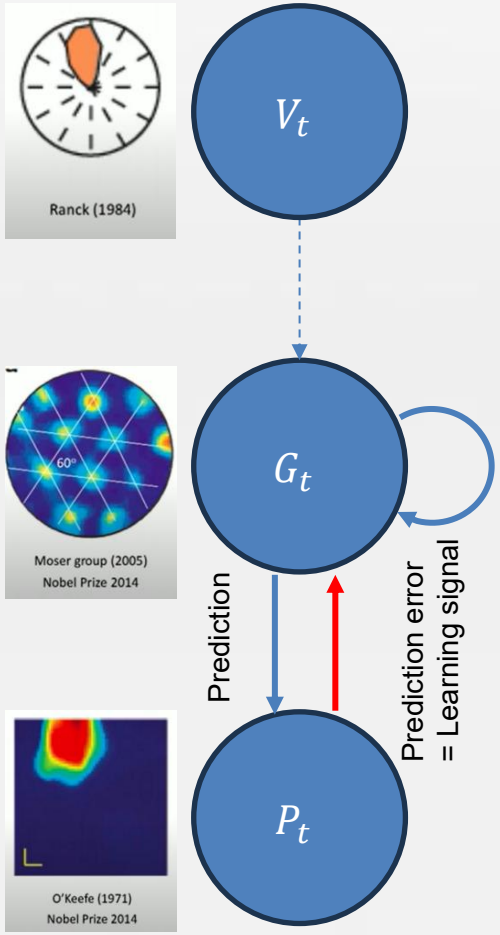
https://arxiv.org/abs/2410.01022



Path integration using gray place cell centers and learned grid cells

Model

Control input



Hidden state

Grid cells = **Map**

Sensory observation

Place cels = **landmarks**

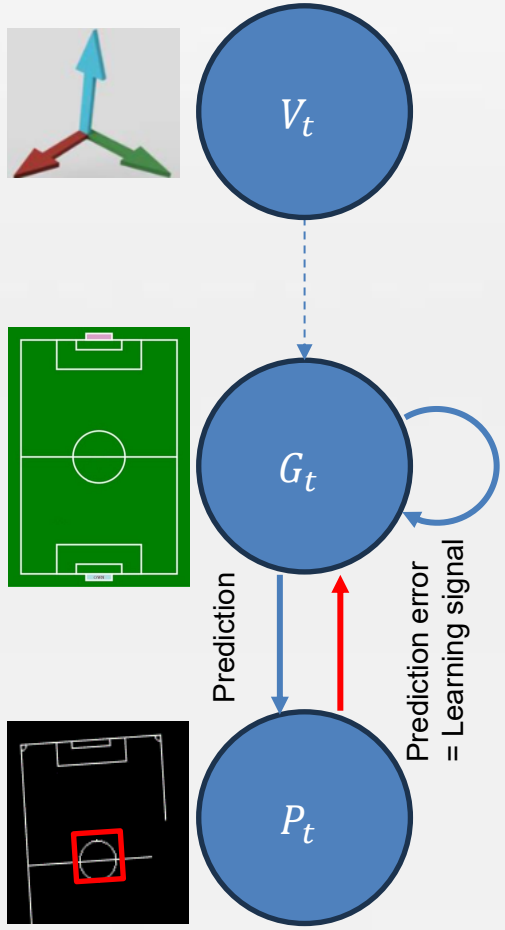
Robot Soccer Self-Localization with Active Inference

Model

Control input - Velocity (Odometry) and Acceleration (IMU)

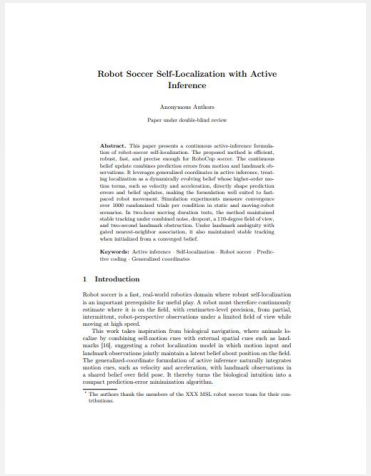
Velocity = $V_t' = \begin{bmatrix} x' \\ y' \\ \theta' \end{bmatrix}_t$ =Odometry

Acceleration = $V_t'' = \begin{bmatrix} x'' \\ y'' \\ \theta'' \end{bmatrix}_t$ =IMU



Hidden state - location on the soccer field

$\tilde{G}_t = \begin{bmatrix} \tilde{x} \\ \tilde{y} \\ \tilde{\theta} \end{bmatrix}_t$ In generalized coordinates $\tilde{*} = \begin{bmatrix} * \\ *' \\ *'' \end{bmatrix} = \begin{bmatrix} position \\ speed\ at\ position \\ acceleration\ at\ position \end{bmatrix}$



Robot Soccer Self-Localization with Active Inference

Anonymous Authors

Paper under double-blind review

Abstract. This paper presents a continuous active-inference formulation of robot-soccer self-localization. The proposed method is efficient, robust, fast, and precise enough for RoboCup soccer. The continuous belief update combines prediction errors from motion and landmark observations. It leverages generalized coordinates in active inference, treating localization as a dynamically evolving belief whose higher-order motion terms, such as velocity and acceleration, directly shape prediction errors and belief updates, making the formulation well suited to fast-paced robot movement. Simulation experiments measure convergence over 1000 randomized trials per condition in static and moving-robot scenarios. In two-hour moving duration tests, the method maintained stable tracking under combined noise, dropout, a 110-degree field of view, and two-sensed landmark obstruction. Under landmark ambiguity with gated nearest-neighbor association, it also maintained stable tracking when initialized from a converged belief.

Keywords: Active inference · Self-localization · Robot soccer · Predictive coding · Generalized coordinates

1 Introduction

Robot soccer is a fast, real-world robotics domain where robust self-localization is an important prerequisite for useful play. A robot must therefore continuously estimate where it is on the field, with centimetre-level precision, from partial, intermittent, robot-perspective observations under a limited field of view while moving at high speed.

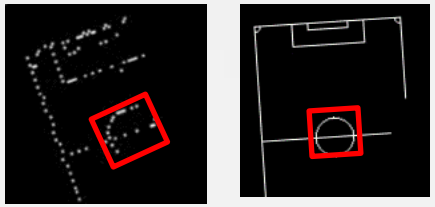
This work takes inspiration from biological navigation, where animals localize by combining self-motion cues with external spatial cues such as landmarks [10], suggesting a robot localization model in which motion input and landmark observations jointly maintain a latent belief about position on the field. The generalized-coordinate formulation of active inference naturally integrates motion cues, such as velocity and acceleration, with landmark observations in a shared belief over field pose. It thereby turns the biological intuition into a compact prediction-error minimization algorithm.

The authors thank the members of the XXX MSL robot soccer team for their contributions.

Observations - Distinctive landmarks

- predict what landmarks to observe where in its sensors

Observation – prediction = sensory prediction error $\varepsilon_y = \begin{bmatrix} \varepsilon_x \\ \varepsilon_y \\ \varepsilon_\theta \end{bmatrix}$



- calculate the error in terms of x, y, θ between observed landmark and expected landmark, for each landmark.

Where am I on the football field?

Football field is known
- Predefined football field with known dimensions
- Given, does not to be learned

Landmarks are known
- Predefined hard-coded place cells
  - Goal (posts)
  - Distinct line patterns (e.g. middle circle)
- Given, does not to be learned

Relation location -> landmarks is known
- Predefined mapping, on any position on the field you can calculate what landmarks to be predicted where
- Given, does not to be learned

To be learned: **location** on the football field
- Requirement: Pose in Field Coordinate System (FCS)
  - x position
  - y position
  - θ angle of the front of the robot

Soccer field

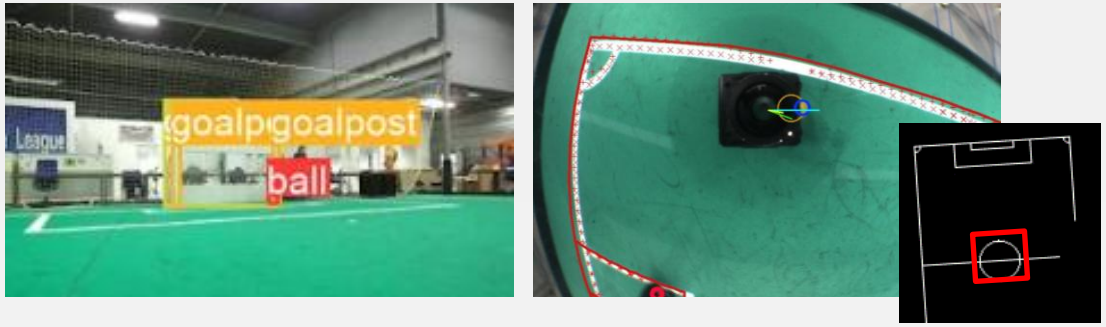
Football field is known
- Predefined football field with known dimensions
- Given, does not to be learned

Landmarks are known
- Predefined hard-coded place cells
  - Goal (posts)
  - Distinct line patterns (e.g. middle circle)
- Given, does not to be learned

Relation location -> landmarks is known
- Predefined mapping, on any position on the field you can calculate what landmarks to be predicted where
- Given, does not to be learned

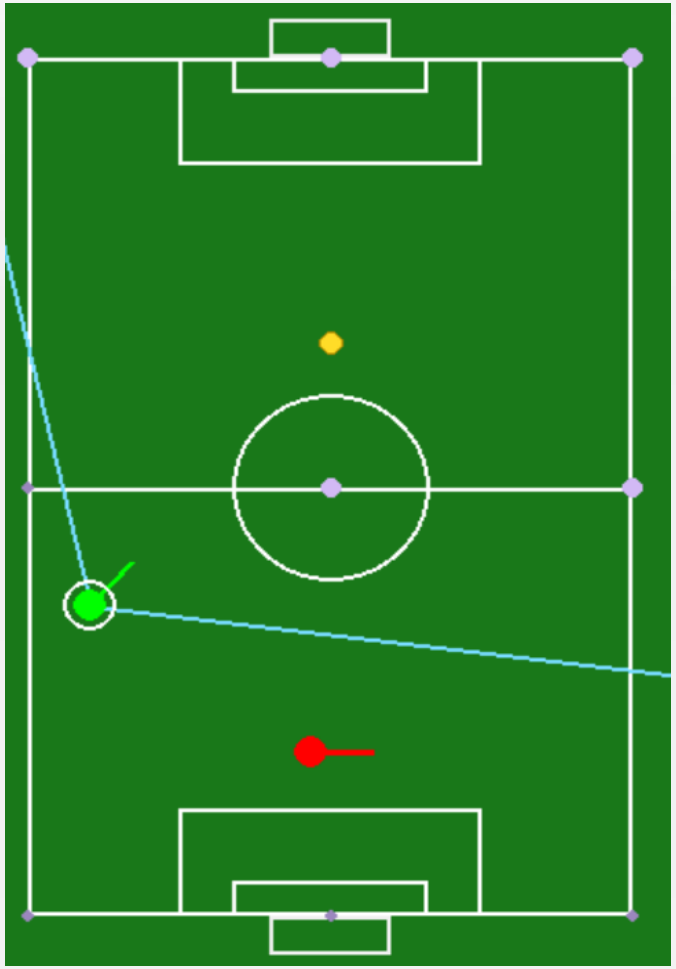
To be learned: **location** on the football field
- Requirement: Pose in Field Coordinate System (FCS)
  - x position
  - y position
  - θ angle of the front of the robot

Landmarks



Needs to be robust, fast, and precise enough for RoboCup soccer

- Fast-moving robots
  - reaching speeds up to 5 m/s and accelerations up to 12 m/s2
- Centimeter-scale tracking
- Robust
  - no robots running out of the field
- Under noisy, partial, intermittent, and ambiguous landmark observations
  - including temporary obstruction
- 110-degree field of view.

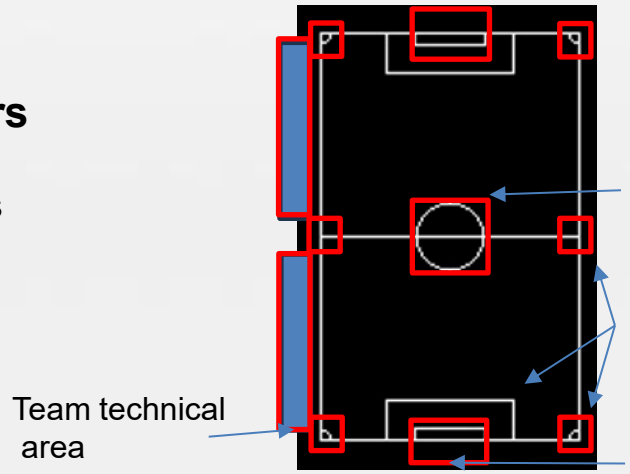


What are landmarks in a robot football field context?

Distinct **landmarks**

- Landmarks on football field are given:
  - Goal (posts)
  - Distinct line patterns (e.g middle circle)
- **The robot predicts what landmarks it expects to observe where in its sensors**
  - Based on location in the field
  - Predictions errors are the learning signals
- Multi-sensor
  - Omni-camera
  - Zed-2 (depth) camera
  - Lidar
  - Etc

Landmarks in Omni-camera
(e.g. distinctive line patterns)

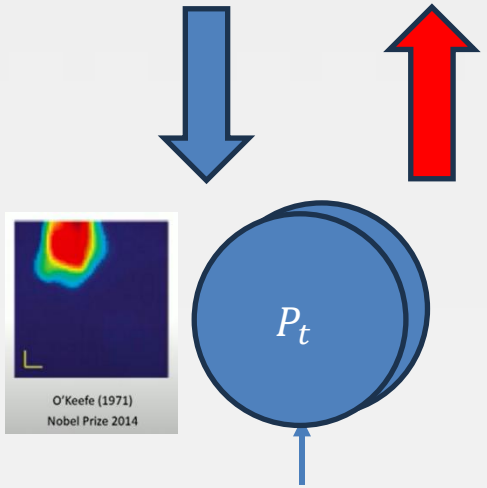


Landmark in Zed-2 camera
(e.g. Yolo goal post detect)



Predicted landmarks in Sensors

Prediction error Learning signal



Sensory observations

What are grid cells in a robot football field context?

Location on/nearby the football field

- Position in Field Coordinate System

$$\tilde{G}_t = \begin{bmatrix} \tilde{x} \\ \tilde{y} \\ \tilde{\theta} \end{bmatrix}_t$$

- In generalized coordinates, tracking of higher order derivatives

$$\tilde{*} = \begin{bmatrix} * \\ *' \\ *'' \end{bmatrix} = \begin{bmatrix} position \\ speed\ at\ position \\ acceleration\ at\ position \end{bmatrix}$$

Control Input (V_t)

Velocity = $V_t' = \begin{bmatrix} x' \\ y' \\ \theta' \end{bmatrix}_t$ = Odometry

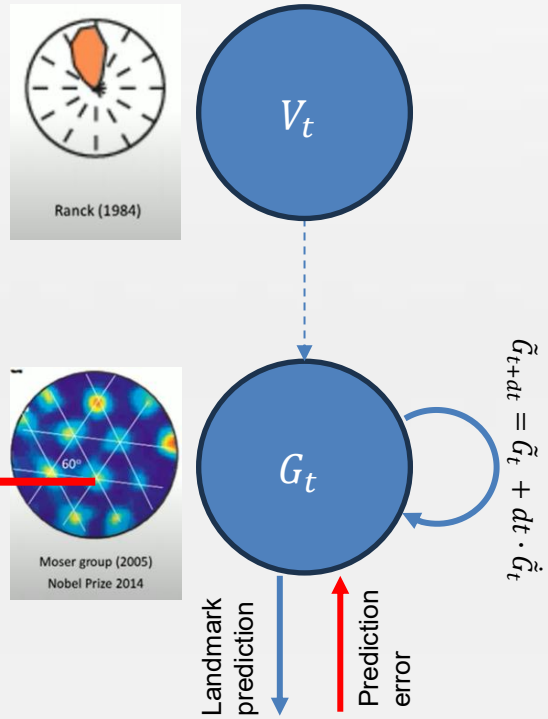
Acceleration = $V_t'' = \begin{bmatrix} x'' \\ y'' \\ \theta'' \end{bmatrix}_t$ = IMU

Design choice for implementation scenario:

A. 1 hidden state
   - Track robot field position with control input and place cell prediction errors

B. 2D dense grid
   - Calculate field position with control input and for all grid cells the place cell prediction errors

C. particle field
   - Option A with multiple possible location hypotheses



Natural Intelligence by Karl Friston

- Neuroscience has produced a candidate which suggests that several brain theories might be unified
- Free-energy principle
- Predictive coding network

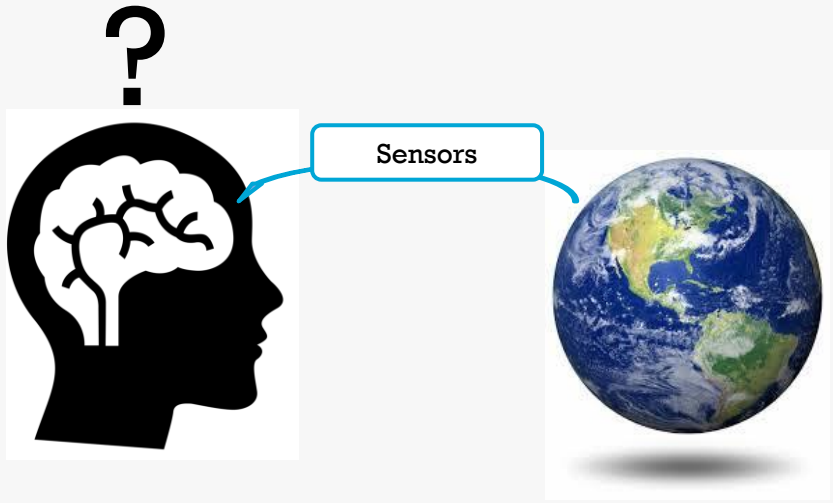
WIRED

The Genius Neuroscientist Who Might Hold the Key to True AI

Karl Friston's free energy principle might be the most all-encompassing idea since the theory of natural selection. But to understand it, you need to peer inside the mind of Friston himself.



THE BRAIN NEEDS TO INFER THE REAL WORLD



The brain is skull-bound and needs to infer the causes of its sensory input (neuronal spikes)

HOW DO YOU EXPERIENCE THE WORLD?



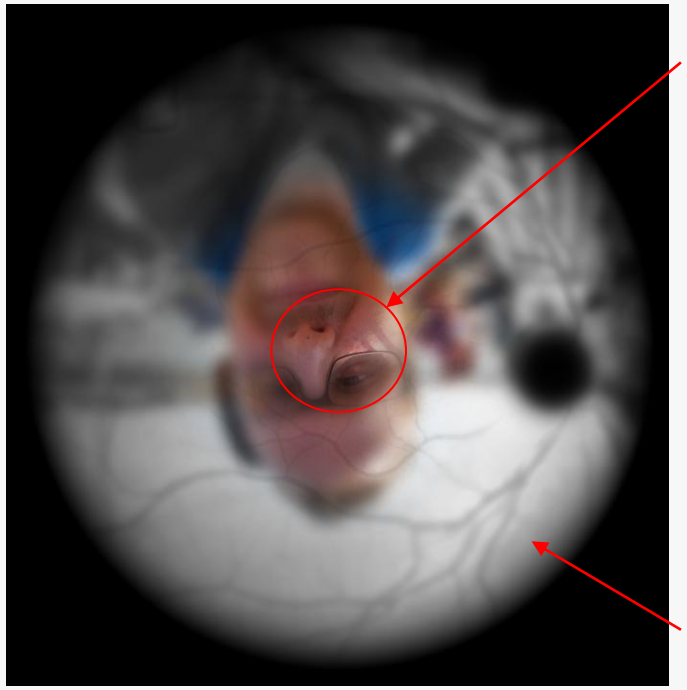
Is the dress
a) black and blue
b) white and gold

What do you see?

Your perception is a "controlled hallucination" (Anil Seth)

HOW DO YOU EXPERIENCE THE WORLD?

Look around you.



sharp, color
(Fovea)

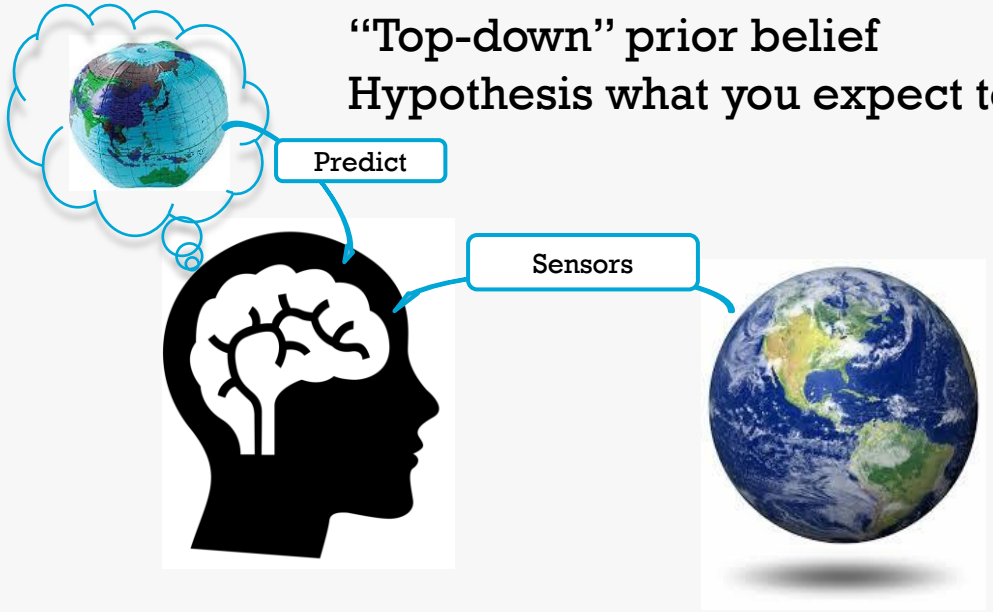
Fuzzy, colorless

What your eyes actually see…

The richness of perceptual experience is your own illusion

Your perception is not just sensory information

INFER THE REAL WORLD



The brain infers the world by encoding model of the world in which it is immersed

HOW DO WE LEARN FROM OBSERVATIONS?

Bayesian inference



Revered Thomas Bayes (1702-1761)

Posterior
probability of hypothesis given sensory data

Likelihood
probability of sensory data given hypothesis

Prior
Hypothesis
Prior belief

$$p(hypothesis\ x|data\ y) = \frac{p(data\ y|hypothesis\ x)\ * p(hypothesis\ x)}{p(data\ y)}$$

Evidence
normalizes the probability, probability of sensory data given all possible causes
Discrete: $\sum_x p(\text{data } y|hypothesis\ x)\ *\ p(\text{hypothesis } x)$

Update our beliefs about the world based on evidence that we observe.

BAYESIAN BRAIN EXAMPLE

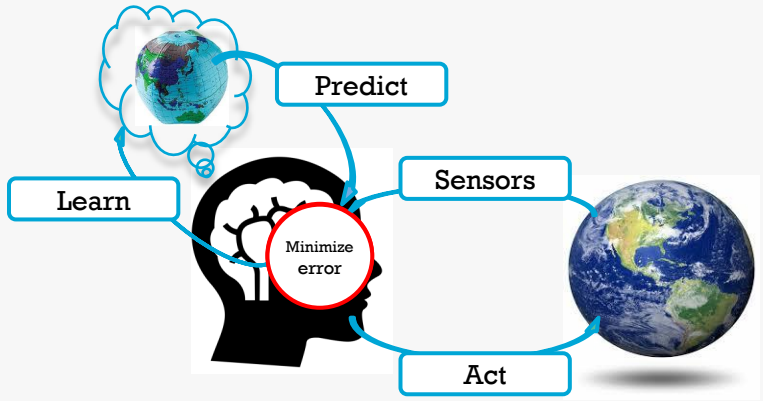


I see cookie crumbs; did she steal a cookie?

$$p(StoleCookie|SeeCrumbs) = \frac{p(SeeCrumbs|StoleCookie) * p(StoleCookie)}{p(SeeCrumbs)}$$

THE BRAIN PREDICTS

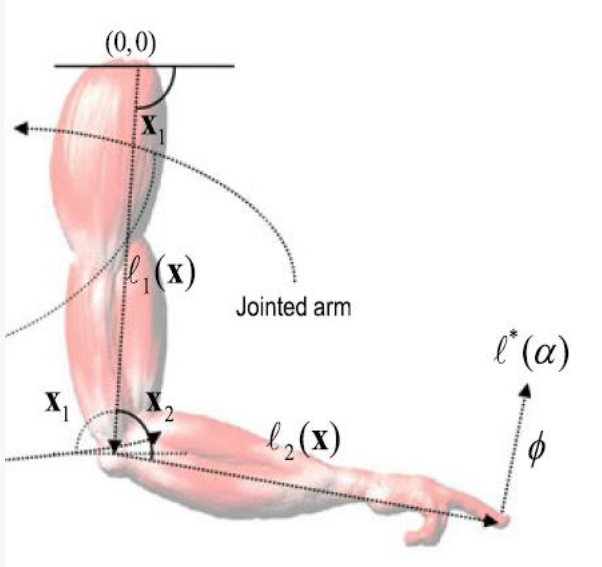
- The skull-bound brain needs to infer the world.
- The brain builds an internal model of the world using Bayesian inference.
- Discrepancies between the prediction of the internal model and the sensory observations result in prediction error.
- The brain aims to minimize the prediction errors (Free Energy) .
- By learning a better internal model or acting upon the environment.



The brain builds an internal model of the world, the world has structure and therefore the brain can predict.

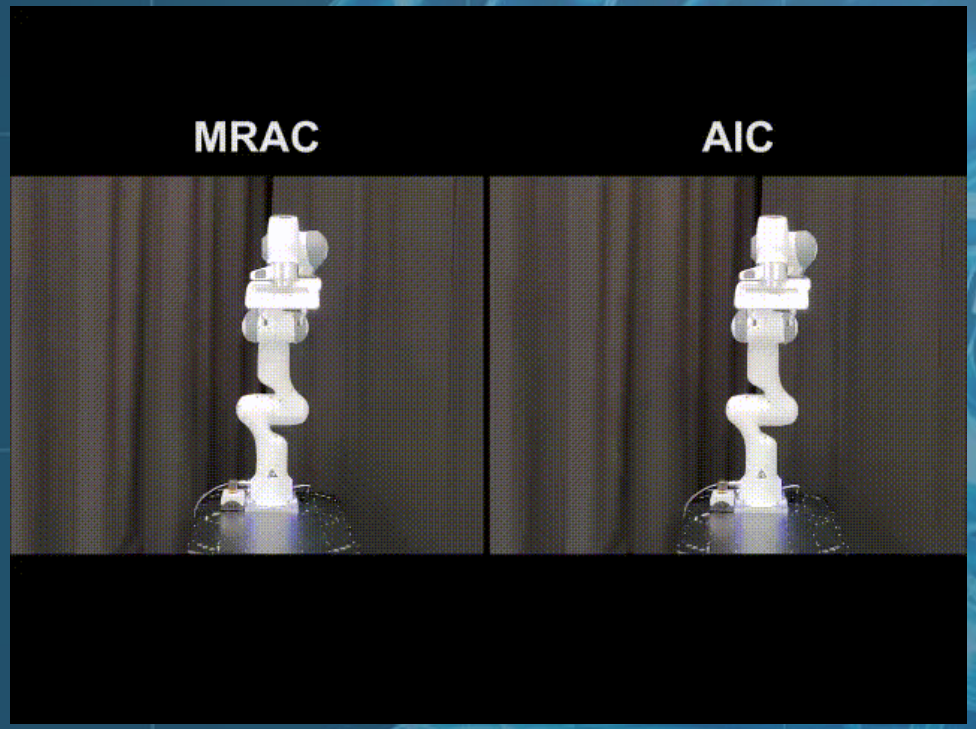
ACTION AS (SELF-FULFILLING) PREDICTION

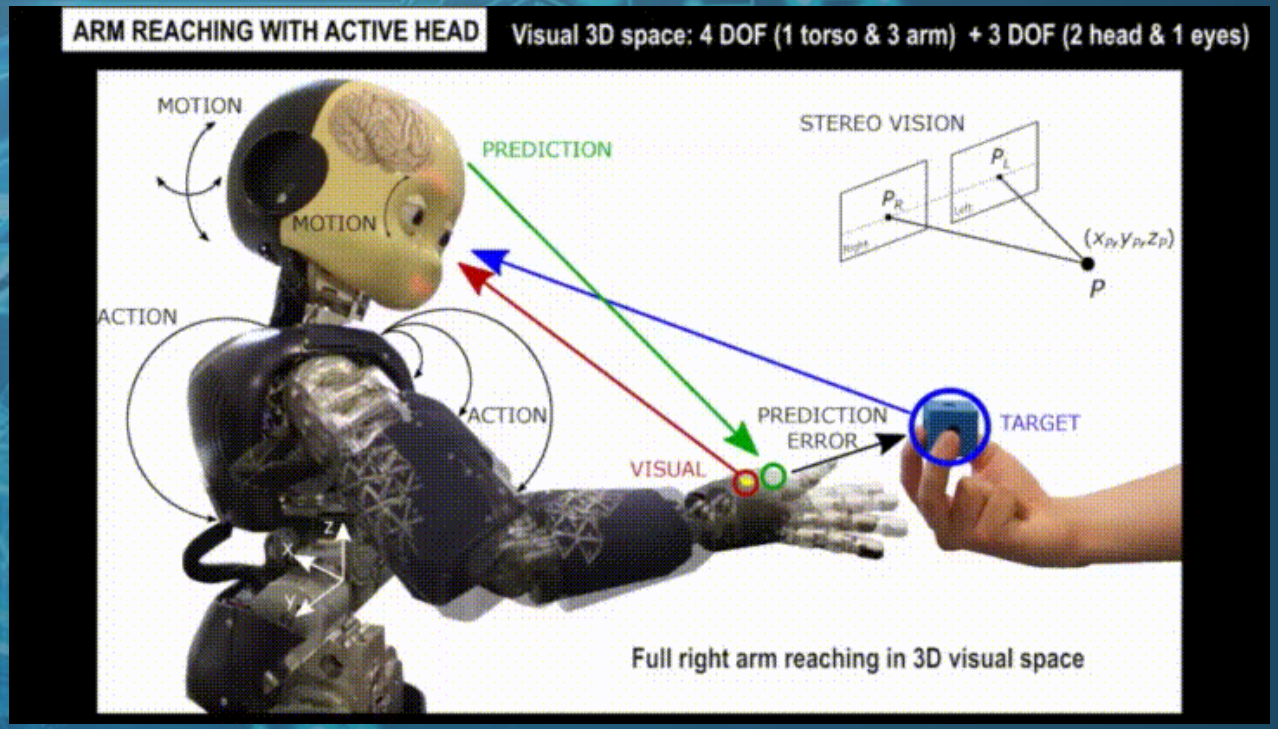
- The "active part" of active inference
- Solving the prediction error by changing the sensory signals to the ones you expect/predict
- Action as classical reflex arcs
- Inference and action are resoled together.
- Perception is an action-based process E.g. eye saccades



Action as a self-fulfilling prophecy

DOES IT WORK?





Robot control room buzzing with predictions of each and every signal

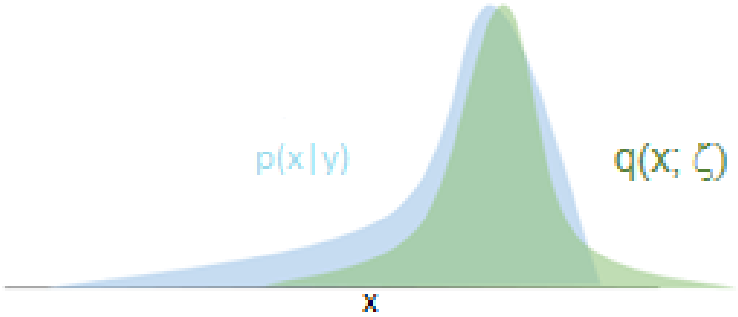
Variational Bayesian Inference

Exact Bayesian inference $p(x|y) = \frac{p(y|x) * p(x)}{p(y)}$ is (computationally) intractable

- How to calculate $p(y)$ given all possible causes $p(y) = \int_x p(y|x) * p(x)$??

approximates the target distribution $p(x|y)$ by q(x; ζ)

- from a family of "nice" probability distributions
  - is specified by its sufficient statistics ζ
  - E.g. for a Gaussian distribution: mean μ, variance σ^2



Using Kullback-Leibler divergence

- measures difference between 2 probability distributions
- Minimize KL with respect to ζ to approximate target probability distribution

$$\underset{\zeta}{\operatorname{argmin}}\, D_{KL}(\, q(x;\zeta\,) \,||\, p(x|y))$$

Conventions used in these slides
x - hidden states
y - are sensory observations

A Short Introduction to Entropy, Cross-Entropy and KL-Divergence

Free energy

$$D_{KL}(\, q(x;\zeta\,) \,||\, p(x|y)) = \int q(x;\zeta\,) \ln\left(\frac{q(x;\zeta\,)}{p(x|y)}\right) dx$$

Definition

$$= \int q(x;\zeta\,) \ln\left(\frac{q(x;\zeta\,)p(y)}{p(x,y)}\right) dx$$

p(x,y)=p(x|y)p(y)

$$= \int q(x;\zeta\,) \ln\left(\frac{q(x;\zeta\,)}{p(x,y)}\right) dx + \int q(x;\zeta\,) \ln(p(y))\, dx$$

ln(ab)=ln(a)+ln(b)

$$= \int q(x;\zeta\,) \ln\left(\frac{q(x;\zeta\,)}{p(x,y)}\right) dx + \ln(p(y)) \int q(x;\zeta\,) dx$$

No x in p(y)

$$= \int q\,(x;\zeta\,) \ln\left(\frac{q(x;\zeta\,)}{p(x,y)}\right) dx + \ln(p(y))$$

By definition $\int q(x;\zeta\,)dx$=1

Free energy $\mathcal{F}$(y,□)

Minimize free energy

$$\underset{\zeta}{\operatorname{argmin}} \mathcal{F}(y,\zeta) = \int q(x;\zeta) \ln\left(\frac{q(x;\zeta)}{p(x,y)}\right) dx$$

Recognition density $q(x;\zeta)$

Hidden states estimations

E,g estimates the current position/depth of Hydar, eg 10 centimeter with a variance of 1 centimeter

Generative density $\mathrm{p(x,y)} = p(y|x) * p(x)$

- $p(x)$: model how a position x changes over time
- $p(y|x)$: Model of how hidden states translate to sensory observations, e.g. for Hydar what temperate belongs to what depth

Free-energy can be evaluated

- it is a function of sensory data and brain states

Free energy can be minimized

- It is an automatable optimization problem

By minimizing the Free Energy

- It learns the hidden states $q(x;\zeta) \approx p(x|y)$ because also $D_{KL}(\, q(x;\zeta) \,||\, p(x|y))$ is minimized
- And it minimizes the free-energy bound on surprise in sensory observations

Minimizing free energy minimizes surprise in sensory observations

$D_{KL} > 0$

$$\mathcal{F}(y, \zeta) = D_{KL}(\, q(x;\zeta) \,||\, p(x|y)) - \ln(p(y)) \geq \ln\left(\frac{1}{p(y)}\right)$$

Surprise

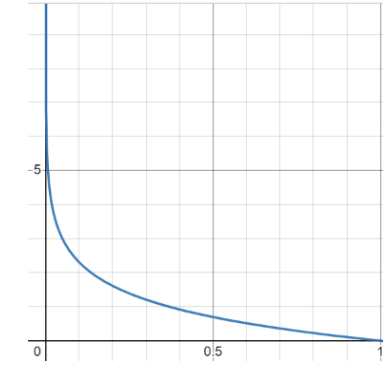
Inverse of the probability

Ln as measuring information density

- Eg 100 options in 2 digits = $\log_{10}(10^2) = 2$
- eg 8 options in binary bits = $\log_2(2^3) = 3$

Why is this important? → Survival !

- To survive, biological agents preserve their physical integrity.
- Biological agents avoid surprising states.
- E.g Hydar is an aquatic animal
  - Hydar in the water is the “unsurprising state”
  - Hydar outside the water will not be observed because it will die
- Agents can avoid surprising exchanges with the world if it minimizes its free-energy because Free energy ≥ surprise



high probability = low surprisal

low probability = high surprisal

Simplify the complex Free Energy integral

Recognition density $q(x;\zeta)$ is simplified by estimating by its mean $q(x;\zeta) \approx \mu$

$$\underset{\mu}{\operatorname{argmin}} \mathcal{F}(y, \mu) \approx -\ln p(\mu, y)$$

If we assume

- Probability distributions are Gaussian
- Recognition density $q(x;\zeta)$ is sharply peaked and estimated by a second order Taylor approximation
- Omit constants (remember the objective is to minimize the Free Energy)
- The variable sets can be partitioned into several independent sets, e.g. temporal scales (the mean field approximation $p(a,b) \approx p(a)p(b)$)

TU Delft

If you want to double check it

The Free energy is defined as:

$$\mathcal{F}(y,\zeta) = \int q(x;\zeta) ln(\frac{q(x;\zeta)}{p(x,y)}) dx$$

since ln(a/b)=ln(a)-ln(b)

and defining the Laplace-encoded Energy as $E(x,y) = -ln\, p(x,y)$

$$= \int q(x;\zeta) E(x,y) dx + \int q(x;\zeta) ln(q(x;\zeta)) dx$$

Under the Laplace assumption the Recognition Density is assumed to be Gaussian, thus:

$$q(x;\zeta) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Substituting this equation in the Free Energy,

$$= \int q(x;\zeta) E(x,y) dx - ln\sqrt{2\pi\sigma^2} - \frac{1}{2})$$

and applying a second order Taylor approximation (derivative information at a point translates to approximation around the point) of E(x, y) around x = μ (non-zero density only at sharp peak μ) allows to re-write the approximation as (note: first order derivative is 0 because peak at μ):

$$\approx E(\mu,y) + \frac{1}{2}(\left[\frac{d^2E}{dx^2}\right]_\mu \sigma^2 - ln 2\pi\sigma^2 - 1))$$

By solving $d\mathcal{F} = 0$ with respect to the variance σ^2 (which optimizes the free-energy), it further simplifies to:

$$\approx E(\mu,y) - \frac{1}{2}(ln 2\pi\sigma^{2*})$$

Where $\sigma^{2*} = \left[\frac{d^2E}{dx^2}\right]_\mu^{-1}$ is the variance that optimizes the Free Energy. It is a constant, which is calculated from the mean μ. In other words, the Free Energy only depends on the Gaussian mean μ and not σ, which significantly simplifies the expression. To further simplify and get rid of the constant (remember that the objective is to minimize the free energy so a constant doesn't matter),

$$\approx E(\mu,y) = -ln\, p(\mu,y)$$

the **Laplace encoded Free Energy** as approximation of the Free Energy is written as:

$$\mathcal{F}(y,\mu) = -ln\, p(\mu,y) \approx \mathcal{F}(y,\zeta)$$

This equation also holds for the multivariate case with $\vec{x} \in \mathbb{R}^n$, $\vec{y} \in \mathbb{R}^q$ and $\vec{\mu} \in \mathbb{R}^n$ (assuming the mean-field approximation, and the multidimensional recognition density is still tightly peaked, see the free energy principle for action and perception: a mathematical review paper for the detailed proof).

For a detailed mathematical review, see paper (77 pages)
the free energy principle for action and perception: a mathematical review (Buckley)

My summary
https://www.kaggle.com/code/charel/learn-by-example-active-inference-in-the-brain-2

How to compute the Free Energy?

Prediction error

$$\varepsilon_y = y - g(\mu_x, \mu_v, \mu_\theta)$$
$$\varepsilon_x = x' - f(\mu_x, \mu_v, \mu_\theta)$$

$$\underset{\mu}{\operatorname{argmin}} \mathcal{F}(y, \mu) = \frac{1}{2}\varepsilon^T \Pi \varepsilon - \frac{1}{2} ln|\Pi|$$

Precision weighted

Generative density $p(x, y) = p(y|x) * p(x)$

- $p(x)$: model how a position x changes over time
- $p(y|x)$: Model of how hidden states translate to sensory observations, e.g. for Hydar what temperate belongs to what depth

Dynamical model is used to predict the dynamics of the world: State-space model

- Things in the world move e.g. speed is connecting positions (spatial) over time (temporal).
- Confer memory to the system. Things do not “teleport” or suddenly become something else.
- Substituting the Gaussian probability densities in the free energy equation
  - $p(x) \quad : \mu_x' = f(\mu_x, \mu_v, \mu_\theta) + \omega_x(\mu_\lambda) = \mathcal{N}(\mu_x' - f(\mu_x, \mu_v, \mu_\theta), \sigma_x^2)$
  - $p(y|x)$: $y = g(\mu_x, \mu_v, \mu_\theta) + \omega_y(\mu_\lambda) = \mathcal{N}(y - g(\mu_x, \mu_v, \mu_\theta), \sigma_y^2)$

μ_x: Hidden states
μ_v: Hidden causal states
μ_θ: Parameters of generative model
μ_λ: Hyperparameters defining noises

$\mathcal{F}$

Again, if you want to double check it

In this simplest example the brain is estimating a single static hidden cause. The generative model is

$x = f(x) + w$ (function of motion)

$y = g(x) + z$ (function of sensory mapping)

This can be re-ordered as

$w = x - f(x)$

$z = y - g(x)$

Since under the Laplace assumption all probability densities are Gaussian, it also applies for w and z

$w = \mathcal{N}(0, \sigma_w^2)$

$z = \mathcal{N}(0, \sigma_z^2)$

Thus the functional form of the generative density (intuition: without the noise x-f(x) is 0 but with the noise it has become a probability distribution in the shape of the added noise)

$p(x) = \mathcal{N}(x - f(x), \sigma_w^2)$ (function of motion)

$p(y \mid x) = \mathcal{N}(y - g(x), \sigma_z^2)$ (function of sensory mapping)

The Laplace encoded Free energy formulation that we minimize with active inference

$$\mathcal{F} = -ln\, p(\mu, y)$$

product rule

$$-ln\, p(y \mid \mu) p(\mu)$$

ln(ab)=ln(a)+ln(b)

$$-ln\, p(y \mid \mu) - ln\, p(\mu)$$

substituting the densities

$$-ln \frac{1}{\sqrt{2\pi\sigma_z^2}} e^{-\frac{(y-g(\mu))^2}{2\sigma_z^2}} - ln \frac{1}{\sqrt{2\pi\sigma_w^2}} e^{-\frac{(\mu-f(\mu))^2}{2\sigma_w^2}}$$

$ln(e^a) = a$

$$\frac{(y-g(\mu))^2}{2\sigma_z^2} + \frac{(\mu-f(\mu))^2}{2\sigma_w^2} - ln \frac{1}{\sqrt{2\pi\sigma_z^2}} - ln \frac{1}{\sqrt{2\pi\sigma_w^2}}$$

$ln(\sqrt{a}) = \frac{1}{2} ln(a)$

$$x\frac{(y-g(\mu))^2}{2\sigma_z^2} + \frac{(\mu-f(\mu))^2}{2\sigma_w^2} + \frac{1}{2} ln\,(2\pi\sigma_z^2) + \frac{1}{2} ln\,(2\pi\sigma_w^2)$$

grouping of the ln equations

$$\frac{(y-g(\mu))^2}{2\sigma_z^2} + \frac{(\mu-f(\mu))^2}{2\sigma_w^2} + \frac{1}{2} ln\,(\sigma_z^2\sigma_w^2) + ln\,(2\pi)$$

Removing the constant (not relevant when lowering Free Energy)

$$\frac{1}{2\sigma_z^2}\varepsilon_y^2 + \frac{1}{2\sigma_w^2}\varepsilon_x^2 + \frac{1}{2} ln\,(\sigma_z^2\sigma_w^2)$$

For a detailed mathematical review, see paper (77 pages)
[the free energy principle for action and perception: a mathematical review (Buckley)]

My summary
https://www.kaggle.com/code/charel/learn-by-example-active-inference-in-the-brain-2

Minimize free energy

Prediction errors

$$\varepsilon = \begin{bmatrix} \varepsilon_y \\ \varepsilon_x \end{bmatrix} = \begin{bmatrix} y - g(\mu_x, \mu_v, \mu_\theta) \\ x' - f(\mu_x, \mu_v, \mu_\theta) \end{bmatrix}$$

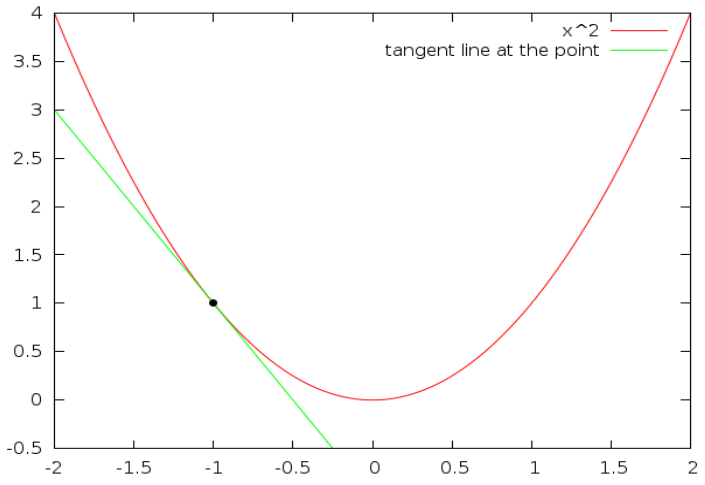
$$\underset{\mu}{\operatorname{argmin}} \mathcal{F}(y, \mu) = \frac{1}{2}\varepsilon^T \Pi \varepsilon - \frac{1}{2} ln|\Pi| \geq \text{surprise}$$

Precisions

$$\Pi = \begin{bmatrix} \Pi_x & 0 \\ 0 & \Pi_y \end{bmatrix} ; \Pi_x = \frac{1}{\sigma_x^2} ; \Pi_y = \frac{1}{\sigma_y^2}$$

Minimize free energy by gradient descent

- Iteratively finding with the lowest Free Energy
- steps proportional to the negative of the partial derivative of the free energy with respect to the variable optimizing



TU Delft

Minimizing the Free Energy

The Free energy is (after a lot of math) the sum of precision weighted squared prediction errors

$$\underset{\mu}{\operatorname{argmin}}\mathcal{F}(y,\mu) = \frac{1}{2}\sum_{i=1}^{\infty}\sum_{j=1}^{P+1}\sum_{k=1}^{n}\left(\Pi_{g(i,j,k)}\varepsilon_{g(i,j,k)}^{2} - \ln\left(\Pi_{g(i,j,k)}\right)\right) + \frac{1}{2}\sum_{i=1}^{\infty}\sum_{j=1}^{P+1}\sum_{k=1}^{q}\left(\Pi_{p(i,j,k)}\varepsilon_{p(i,j,k)}^{2} - \ln\left(\Pi_{p(i,j,k)}\right)\right)$$

By finding 9 parameters $\tilde{\mu}_{gx}, \tilde{\mu}_{gy}, \tilde{\mu}_{g\theta}$ to minimize 69 prediction errors

By gradient descent on the Free Energy

Grid cell (Motion) prediction errors

- $\tilde{\varepsilon}_g$ = grid cell motion prediction error $= D\tilde{\mu}_g - \tilde{f}(\tilde{\mu}_g)$
- $\Pi_{g(i,j,k)}$ = *precision grid cell prediction error*

3 grid cell hidden states:
- $\tilde{\mu}_{gx}, \tilde{\mu}_{gy}, \tilde{\mu}_{g\theta}$

3 generalized coordinates:
- $position, speed, acceleration$

1 level:
- *No hierarchy*

\+

9 precision weighted Grid cell motion prediction errors to be minimized

Place cell (sensory) prediction errors

- $\tilde{\varepsilon}_p$ = place cell prediction error $= \tilde{y} - \tilde{g}(\tilde{\mu}_g)$
- $\Pi_{p(i,j,k)}$ = *precision place cell prediction error*

Approx. 20 place cells/landmarks:
- Calculate separate errors for x, y, θ

No generalized coordinates:
- $only\ position$

1 level:
- *No hierarchy*

\+

60 precision weighted sensory observation prediction errors to be minimized

Gradient descent on the Free Energy

Minimize the Free energy by gradient descent on the Free Energy

$$\dot{\tilde{\mu}}_g = D\tilde{\mu}_g - \partial_{\tilde{\mu}_g}\mathcal{F}(\mathrm{y},\ \mu)$$

The term $D\tilde{\mu}_{gx}$ is due to the generalized coordinates. When the true motion equals the belief of the motion ($\dot{\tilde{\mu}}_{gx} = D\tilde{\mu}_{gx}$), the gradient should reach zero ($\dot{\tilde{\mu}}_{gx}$-$D\tilde{\mu}_{gx} = \partial_{\tilde{\mu}_{gx}}\mathcal{F}$ =0)

$$\dot{\tilde{\mu}}_g = D\tilde{\mu}_g - \left(\partial_{\tilde{\mu}_g}\tilde{\varepsilon}_g^T \cdot \tilde{\Pi}_g\tilde{\varepsilon}_g + \partial_{\tilde{\mu}_g}\sum_{landmarks}\tilde{\varepsilon}_p^T \cdot \tilde{\Pi}_p\tilde{\varepsilon}_p\right)$$

$$\tilde{\varepsilon}_g = D\tilde{\mu}_g - \tilde{f}(\tilde{\mu}_g)$$
$$\tilde{\varepsilon}_p = \tilde{y} - \tilde{g}(\tilde{\mu}_g)$$

$$\dot{\tilde{\mu}}_{gx} = D\tilde{\mu}_g - \left(\left(D - \partial_{\tilde{\mu}_g}\tilde{f}(\tilde{\mu}_g)\right)^T \Pi_g\tilde{\varepsilon}_g + \sum_{landmarks}\left(-\partial_{\tilde{\mu}_g}\tilde{g}(\tilde{\mu}_g)\right)^T \tilde{\Pi}_p\tilde{\varepsilon}_p\right)$$

$$\dot{\tilde{\mu}}_g = D\tilde{\mu}_g - D^T\tilde{\Pi}_g\tilde{\varepsilon}_g + \left(\partial_{\tilde{\mu}_g}\tilde{f}(\tilde{\mu}_g)\right)^T \tilde{\Pi}_g\tilde{\varepsilon}_g + \sum_{landmarks}\left(\partial_{\tilde{\mu}_g}\tilde{g}(\tilde{\mu}_{gx})\right)^T \tilde{\Pi}_p\tilde{\varepsilon}_p$$

Updating $\tilde{\mu}_{gx}$ with the forward Euler method : $\tilde{\mu}_{gx}(t+\Delta t) = \tilde{\mu}_{gx}(t) + \Delta t \cdot \dot{\tilde{\mu}}_{gx}$

The notation $\dot{x} = \frac{\partial}{\partial t}$ is the time derivative of x

The notation $\partial_x = \frac{\partial}{\partial x}$ is the partial derivative towards x; e.g. $\partial_{\tilde{\mu}_x}\tilde{\varepsilon}_x^T\Pi_x\varepsilon_x = \frac{\partial\tilde{\varepsilon}_x^T}{\partial_{\tilde{\mu}_x}}\Pi_x\varepsilon_x$

Gradient descent on the Free Energy

Minimize the Free energy by gradient descent on the Free Energy, example $\tilde{\mu}_{gx}$:

$$\dot{\tilde{\mu}}_{gx} = D\tilde{\mu}_{gx} - D^T\tilde{\Pi}_{gx}\tilde{\varepsilon}_{gx} + \left(\partial_{\tilde{\mu}_g}\tilde{f}(\tilde{\mu}_g)_x\right)^T\tilde{\Pi}_{gx}\tilde{\varepsilon}_{gx} + \sum_{landmarks}\left(\partial_{\tilde{\mu}_g}\tilde{g}(\tilde{\mu}_g)_x\right)^T\tilde{\Pi}_{px}\tilde{\varepsilon}_{px}$$

- $D\tilde{\mu}_{gx} = \begin{bmatrix}0 & 1 & 0\\ 0 & 0 & 1\\ 0 & 0 & 0\end{bmatrix}\begin{bmatrix}\mu_{gx}\\ \mu'_{gx}\\ \mu''_{gx}\end{bmatrix} = \begin{bmatrix}\mu'_{gx}\\ \mu''_{gx}\\ 0\end{bmatrix}$ $where\ \tilde{\mu}_{gx}$ is the belief of the x position/speed/acceleration in field coordinates (FCS),



- $\tilde{\varepsilon}_{gx} = D\tilde{\mu}_{gx} - \tilde{f}(\tilde{\mu}_g)_x = \begin{bmatrix}\mu'_{gx}\\ \mu''_{gx}\\ 0\end{bmatrix} - f_{\text{RCS_2_FCS}}\begin{bmatrix}V'_x\ odometry\\ V''_x\ IMU\\ 0\end{bmatrix}$ = motion belief (in FCS) – predicted motion (in FCS)
  - $\tilde{f}(\tilde{\mu}_g) =$ function of motion: estimating the velocity & acceleration, in our case by using the odometry & IMU.
  - Both in FCS
  - Humanoids/Omi wheels can independently move in x, y or θ directions. Hidden state field positions calculations are factorized
- $\tilde{\Pi}_{gx} = \begin{bmatrix}\Pi_{Odo} & 0 & 0\\ 0 & \Pi_{IMU} & 0\\ 0 & 0 & 0\end{bmatrix}$ the precision of the grid cell prediction errors, in our case the given precision of the Odometry and IMU.
- $\left(\partial_{\tilde{\mu}_g}\tilde{f}(\tilde{\mu}_g)_x\right)^T = 0$ The motion is not dependent of the x position/speed/acceleration on the field location $\tilde{\mu}_{gx}$

Gradient descent on the Free Energy - continued

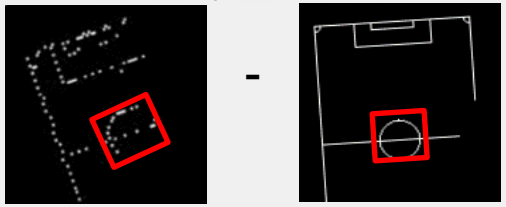
Minimize the Free energy by gradient descent on the Free Energy, example $\tilde{\mu}_{gx}$:

$$\dot{\tilde{\mu}}_{gx} = D\tilde{\mu}_{gx} - D^T\tilde{\Pi}_{gx}\tilde{\varepsilon}_{gx} + \left(\partial_{\tilde{\mu}_g}\tilde{f}(\tilde{\mu}_g)_{\mathrm{x}}\right)^T\tilde{\Pi}_{gx}\tilde{\varepsilon}_{gx} + \sum_{landmarks}\left(\partial_{\tilde{\mu}_g}\tilde{g}(\tilde{\mu}_g)_{\mathrm{x}}\right)^T\tilde{\Pi}_{px}\tilde{\varepsilon}_{px}$$

- $\tilde{\varepsilon}_{px} = \tilde{y}_{\mathrm{x}} - \tilde{g}(\tilde{\mu}_g)_{\mathrm{x}}$ = Place cell sensory prediction error = sensory **observation** landmark – sensory **prediction** landmark
  - No generalized coordinates, only top row / positions
  - $g(\mu_g)$ is the translation from robot position (in μ_g in FCS) to sensory observations (landmarks in RCS).
  - $\tilde{\varepsilon}_{px} = \begin{bmatrix} y_x \\ 0 \\ 0 \end{bmatrix} - \begin{bmatrix} g(\mu_g)_{\mathrm{x}} \\ 0 \\ 0 \end{bmatrix}$ = Observed $_{\text{landmark}}$ – Predicted $_{\text{landmark}}$ =

    •X position centre of landmark
    •In RCS

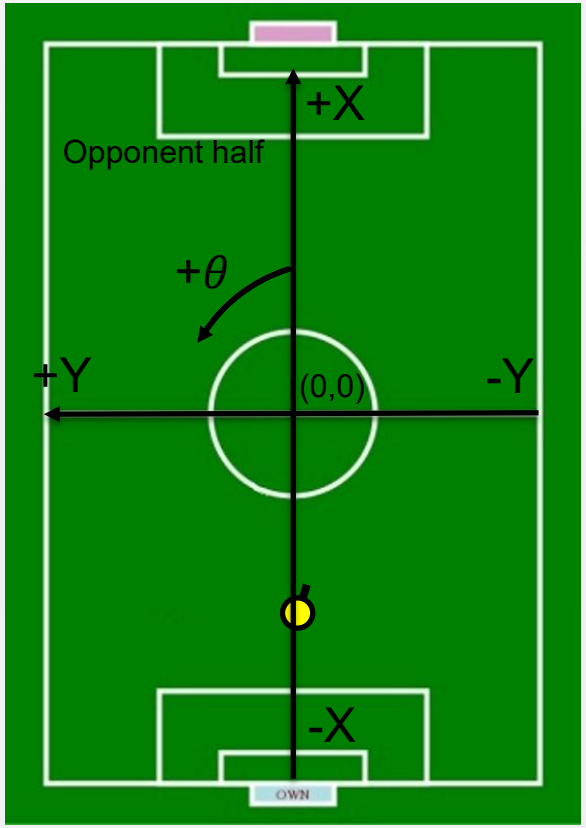
    •X position centre of landmark
    •In RCS

    
    -

- $\Pi_{px} = \begin{bmatrix} \mathrm{mr}_{\mathrm{y}} \cdot \Pi_{camera} & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$ is the precision of the sensory place cell prediction errors,
  - Precision of (the different types of) the camera (Π_{camera})
  - Precision distance modifier (closer by observation more precise) (mr_{y})
- $\partial_{\tilde{\mu}_g}\tilde{g}(\tilde{\mu}_g)$= Jacobian of $g(\mu_g)$ the matrix of partial derivatives that describes how small changes in the robot state $\mathrm{x}, \mathrm{y}, \theta$ affect the predicted sensory observations of landmarks . Note that changes in x but also y, θ affect the predicted x coordinate of the landmark

Translation from robot position (in FCS) to sensory observations of landmarks (in RCS).

Field Coordinate System (FCS)



- The 'θ' in radians with 0 towards enemy goal.

Translation

Input (in FCS)

- Position robot = $\begin{bmatrix} x_r \\ y_r \\ \theta_r \end{bmatrix}$
- Position landmark = $\begin{bmatrix} x_l \\ y_l \end{bmatrix}$

Output (in RCS)

- Where do I expect where to see the place cell / landmark?

$$\begin{bmatrix} x_{pl} \\ y_{pl} \\ \theta_{pl} \end{bmatrix} = \begin{bmatrix} \cos\theta_r\,(x_l - x_r) + \sin\theta_r\,(y_l - y_r) \\ -\sin\theta_r\,(x_l - x_r) + cos\,\theta_r\,(y_l - y_r) \\ atan2(y_{pl}, x_{pl}) \end{bmatrix}$$

- And the Jacobian (derivative is)

$$J_g = \begin{bmatrix} -\cos\theta_r & -\sin\theta_r & y_{pl} \\ \sin\theta_r & -\cos\theta_r & -x_{pl} \\ \dfrac{y_l - y_r}{(x_l - x_r)^2 + (y_l - y_r)^2} & -\dfrac{x_l - x_r}{(x_l - x_r)^2 + (y_l - y_r)^2} & -1 \end{bmatrix}$$

Robot Coordinate System (RCS)



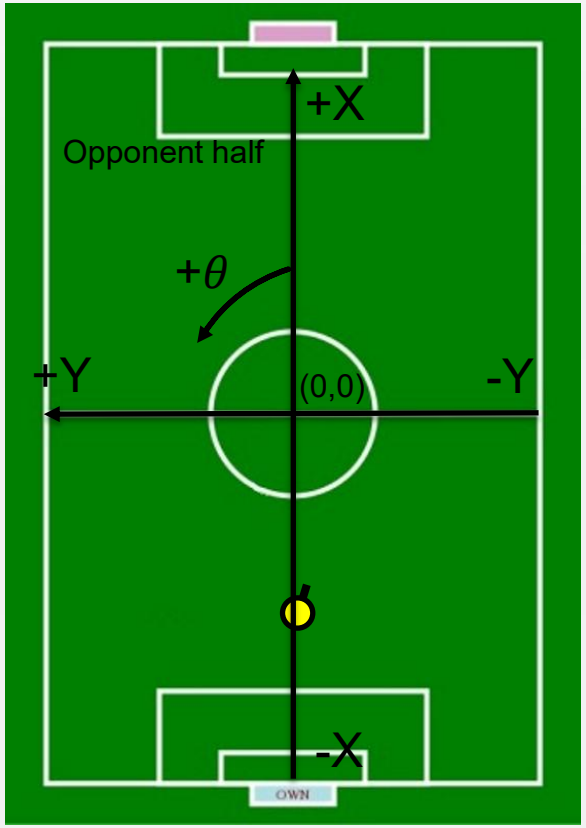


- The 'θ' in radians with 0 straight forward and positive to the left

Note: this coordinate system is based on industry standard ROS. In the Robotsports library the x,y,angle are defined differently and a translation function is made available

Translation from robot motion in FCS to RCS

Field Coordinate System (FCS)



- The 'θ' in radians with 0 towards enemy goal.

Translation

Input (in FCS)

- motion robot = $\begin{bmatrix} x'_{fcs} \\ y'_{fcs} \\ \theta'_{fcs} \end{bmatrix}$ or $\begin{bmatrix} x''_{fcs} \\ y''_{fcs} \\ \theta''_{fcs} \end{bmatrix}$ and angle θ_{fcs}

Output (in RCS)

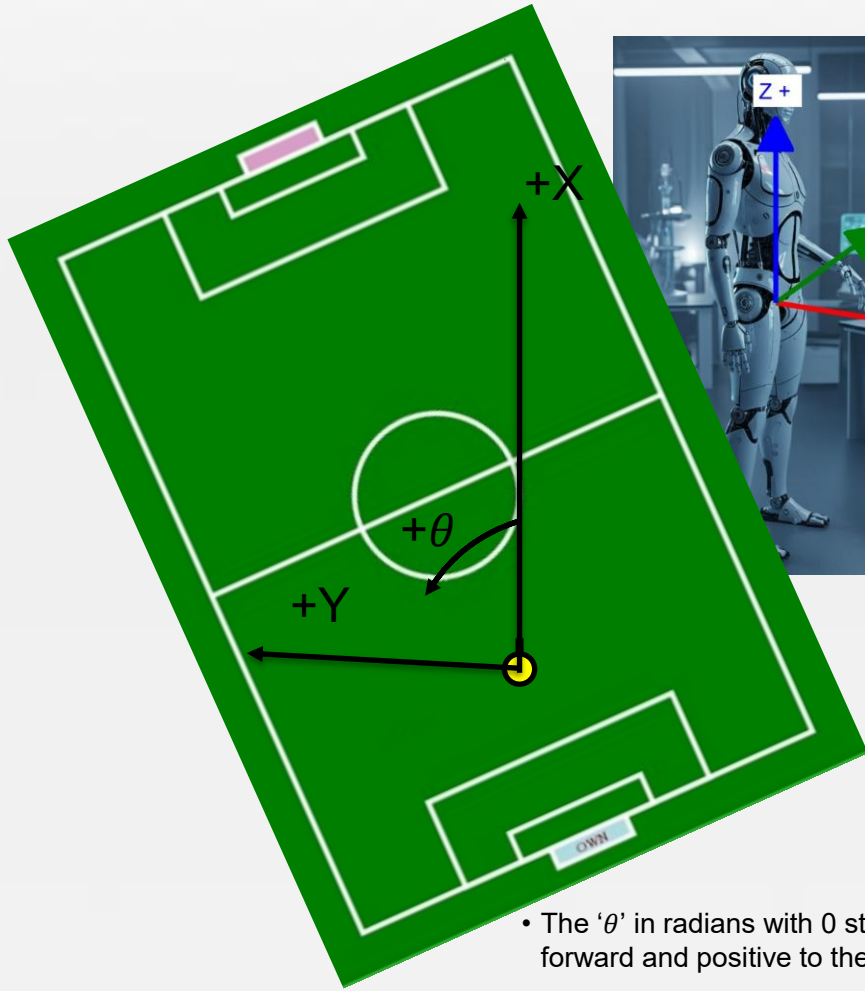
- What is the velocity from Robot perspective (similar for acceleration)

$$\begin{bmatrix} x'_{rcs} \\ y'_{rcs} \\ \theta'_{rcs} \end{bmatrix} = \begin{bmatrix} \cos\theta_{fcs} x'_{fcs} + \sin\theta_{fcs} y'_{fcs} \\ -\sin\theta_{fcs} x'_{fcs} + cos\,\theta_{fcs} y'_{fcs} \\ \theta'_{fcs} \end{bmatrix}$$

- And the Jacobian (derivative is)

$$J_g = \begin{bmatrix} \cos\theta_{fcs} & \sin\theta_r & 0 \\ -\sin\theta_{fcs} & \cos\theta_r & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Robot Coordinate System (RCS)



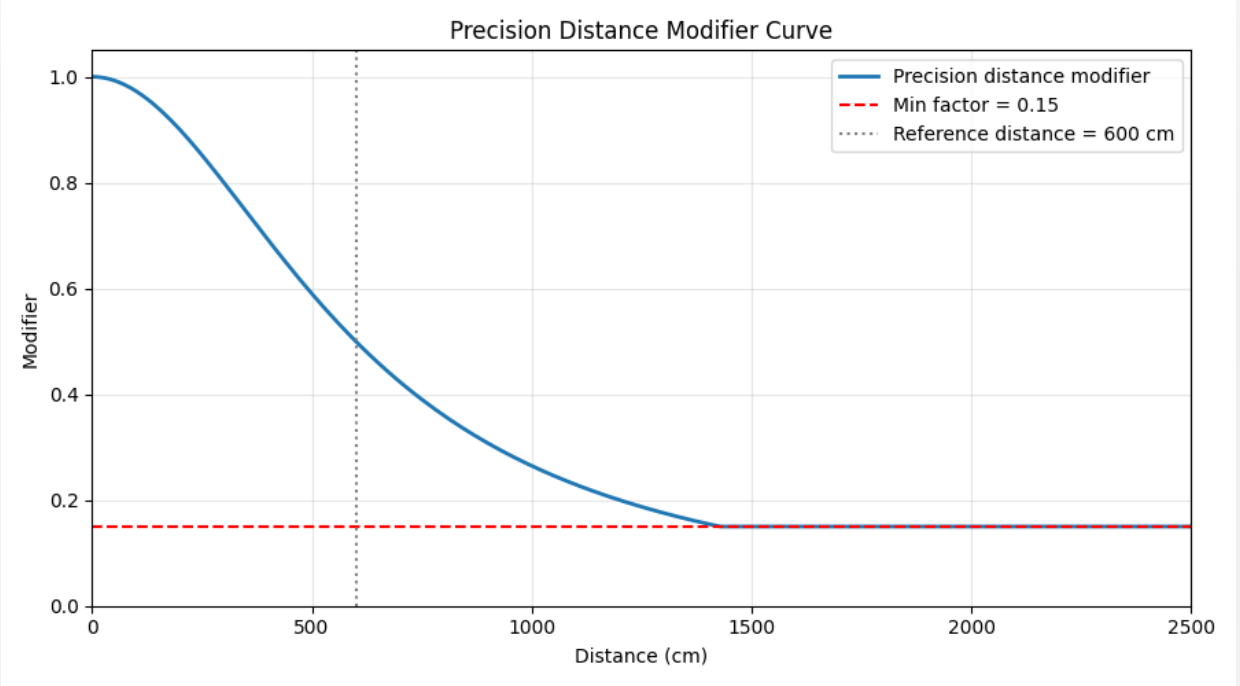
- The 'θ' in radians with 0 straight forward and positive to the left

Note: this coordinate system is based on industry standard ROS. In the Robotsports library the x,y,angle are defined differently and a translation function is made available

Precision distance modifier

Why a precision distance modifier

- Distant landmark observation are less precise (e. In omni camera's the pixel/.. Ratio changes
- Small angle errors result in larger prediction errors in distant place cell



Calculating the precision distance modifier

- Distance is $\mathrm{d}_\iota = \sqrt[2]{g(\mu_g)_\mathrm{x}{}^2 + g(\mu_g)_\mathrm{y}{}^2}$
- Distance modifier $\rho_\iota = \max\left(\frac{1}{1+\left(\frac{\mathrm{d}_\iota}{600}\right)^2}, 0.15\right)$
- Where
  - minimum factor = 0.15
  - Reference distance = 600 cm

Gradient descent on the Free Energy – resulting in

All simple straightforward fast matrix calculations

Resulting in:

$$\begin{bmatrix}\tilde{\mu}_{gx}\\ \tilde{\mu}_{gy}\\ \tilde{\mu}_{g\theta}\end{bmatrix}_{t+\Delta t} = \begin{bmatrix}\tilde{\mu}_{gx}\\ \tilde{\mu}_{gy}\\ \tilde{\mu}_{g\theta}\end{bmatrix}_{t} + \Delta t \cdot \begin{bmatrix}\dot{\tilde{\mu}}_{gx}\\ \dot{\tilde{\mu}}_{gy}\\ \dot{\tilde{\mu}}_{g\theta}\end{bmatrix}_{t}$$

where:

$$\dot{\tilde{\mu}}_{gx} = \begin{bmatrix}\mu'_{gx}\\ \mu''_{gx}\\ 0\end{bmatrix} - \begin{bmatrix}0\\ \eta^{\mathrm{v}}_{\mathrm{xy}}\Pi_{Odo}\left(\mu'_{gx} - f(V'_x\ odometry)\right)\\ \eta^{\mathrm{a}}_{\mathrm{xy}}\Pi_{IMU}\left(\mu''_{gx} - f(V''_x\ IMU)\right)\end{bmatrix} - \sum_{l}\begin{bmatrix}\eta^{\mathrm{c}}_{\mathrm{xy}}\Pi_{cam}\rho_{\mathrm{l}}{\mathrm{J}_{\mathrm{gx}}}^{\mathrm{T}}\left(g(\mu_g)_{\mathrm{x}} - y_x\right)\\ 0\\ 0\end{bmatrix}$$

$$\dot{\tilde{\mu}}_{gy} = \begin{bmatrix}\mu'_{gy}\\ \mu''_{gy}\\ 0\end{bmatrix} - \begin{bmatrix}0\\ \eta^{\mathrm{v}}_{\mathrm{xy}}\Pi_{Odo}\left(\mu'_{gy} - f(V'_y\ odometry)\right)\\ \eta^{\mathrm{a}}_{\mathrm{xy}}\Pi_{IMU}\left(\mu''_{gy} - \mathrm{f}(V''_y\ IMU)\right)\end{bmatrix} - \sum_{l}\begin{bmatrix}\eta^{\mathrm{c}}_{\mathrm{xy}}\Pi_{cam}\rho_{\mathrm{l}}{\mathrm{J}_{\mathrm{gy}}}^{\mathrm{T}}\left(g(\mu_g)_{\mathrm{y}} - y_y\right)\\ 0\\ 0\end{bmatrix}$$

$$\dot{\tilde{\mu}}_{g\theta} = \begin{bmatrix}\mu'_{g\theta}\\ \mu''_{g\theta}\\ 0\end{bmatrix} - \begin{bmatrix}0\\ \eta^{\mathrm{v}}_{\theta}\Pi_{Odo}\left(\mu'_{g\theta} - f(V'_\theta\ odometry)\right)\\ \eta^{\mathrm{a}}_{\theta}\Pi_{IMU}\left(\mu''_{g\theta} - f(V''_\theta\ IMU)\right)\end{bmatrix} - \sum_{l}\begin{bmatrix}\eta^{\mathrm{c}}_{\theta}\Pi_{cam}\rho_{\mathrm{l}}{\mathrm{J}_{\mathrm{g}\theta}}^{\mathrm{T}}\left(g(\mu_g)_{\theta} - y_\theta\right)\\ 0\\ 0\end{bmatrix}$$

←speed
← Acceleration
← Jerk

- If you go too fast, lower acceleration
- Precision weighted

- If you are too far, slow-down
- Precision weighted

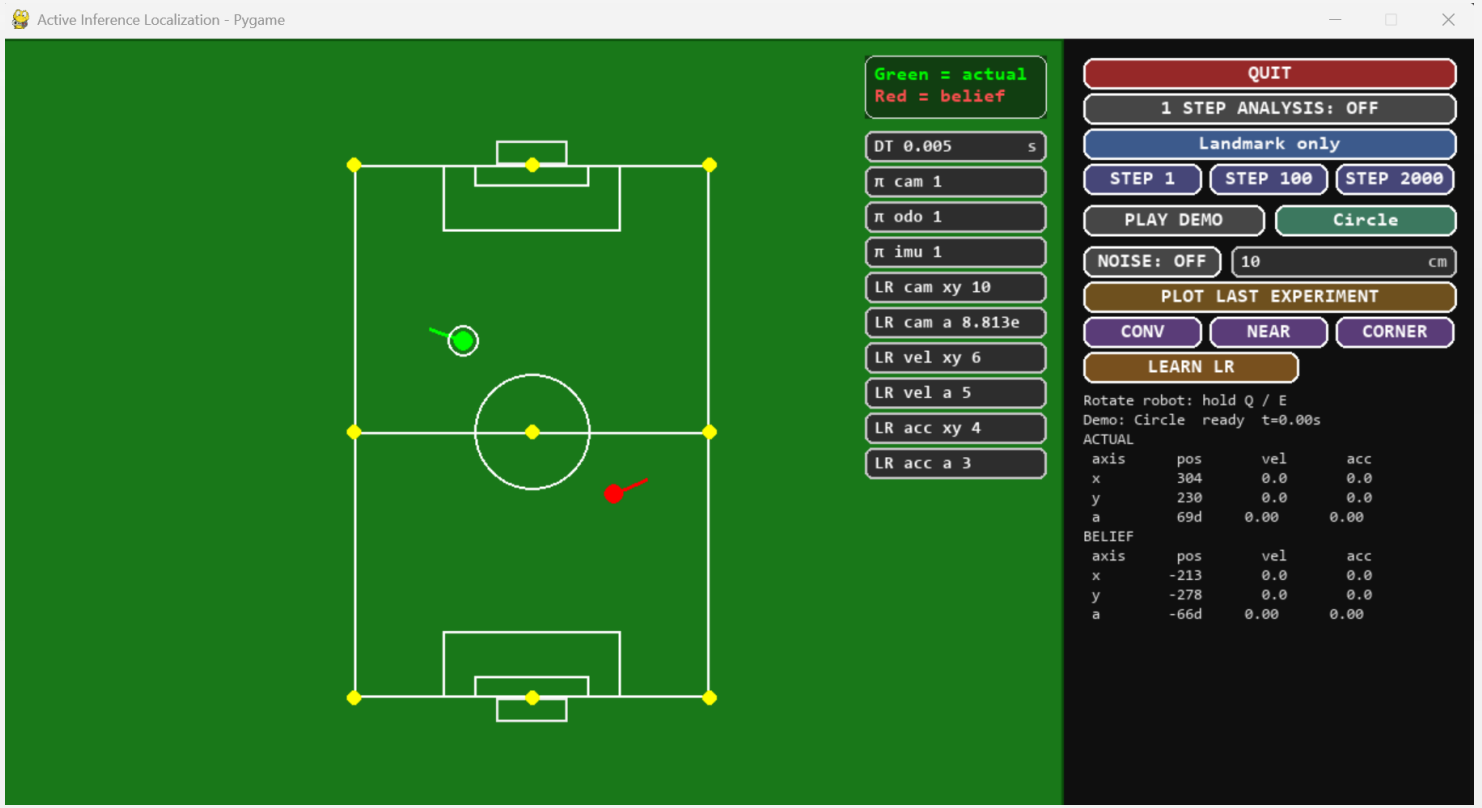
Proof of concept

1. Stationary landmark prediction error minimization with 360 deg view
   - ✓ Works
   - ✓ 100% convergence
   - ✓ Including noise
   - ✓ Learning: precision distance modifier, needs 2 or more landmarks

2. Moving prediction error minimization (landmark + motion) with 360 deg view
   - ✓ Works
   - ✓ 100% convergence
   - ✓ Including noise

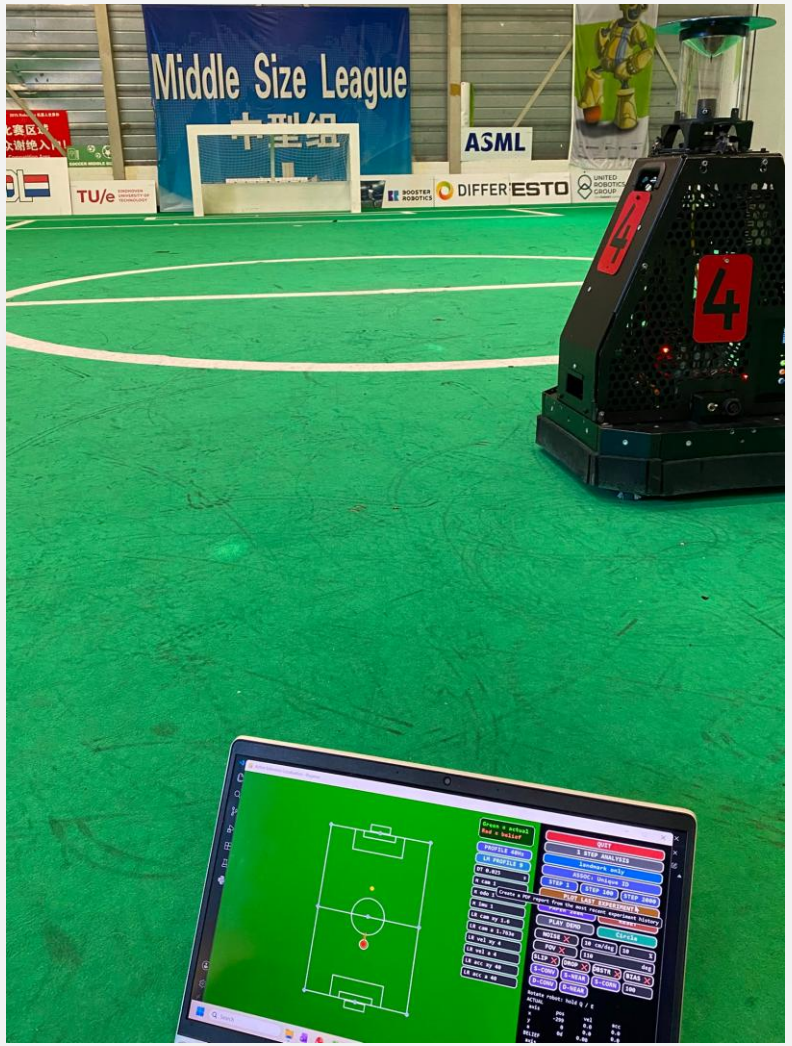
3. Moving prediction error minimization (landmark + motion) with 110 deg view

4. Non-unique landmarks



Next step

Test it on real robots




VDL ROBOTSPORTS
NETHERLANDS

Gradient descent on the Free Energy – resulting in

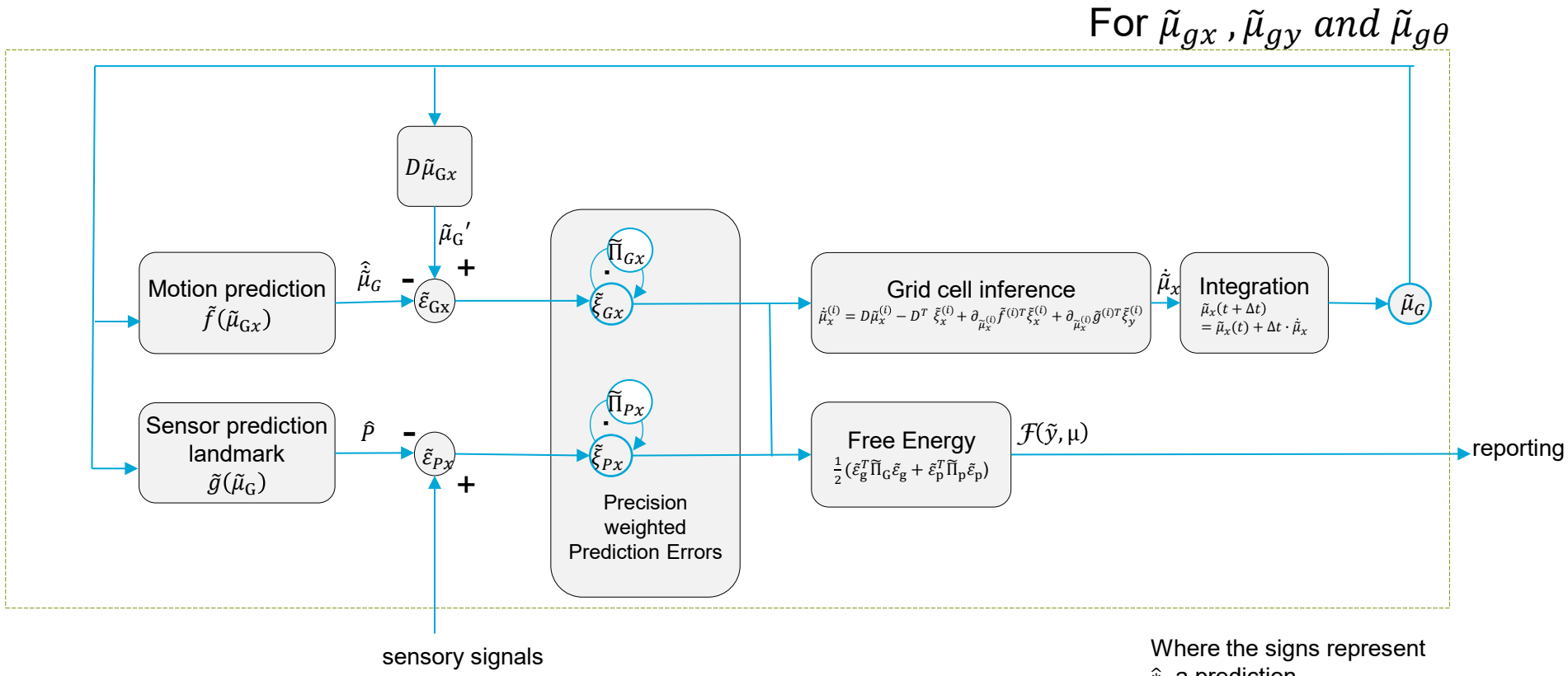
Resulting in:

Dt per sensory stream, treat them while incoming (as long as we have timestamps)
Precision per camera and even per area of camera (omni pixels)

Active inference flow diagram

One active inference capsule with the capability to act upon the environment

Prediction | Prediction error | Prediction error minimization



Where the signs represent
$\hat{*}$ a prediction
$\dot{*}$ a motion
$*'$ an estimated motion
$\tilde{*}$ generalised coordinates

The posterior cingulate region, where invariant-location cells dwell, is closely connected to a site called the parahippocampal gyrus (next to the hippocampus), where “place cells” are found.”
“Fascinatingly, these neurons demonstrably encode where the animal thinks it is.”
“If a rat is ‘teleported’ by suddenly switching the colors of the floor, walls, and ceiling so that they resemble another familiar room, place cells in the hippocampus briefly oscillate between the two interpretations, then settle into a firing pattern appropriate to the illusory room.

MSL Field coordinate system, will be replaced by ROS coordinate system

Field Coordinate System (FCS)

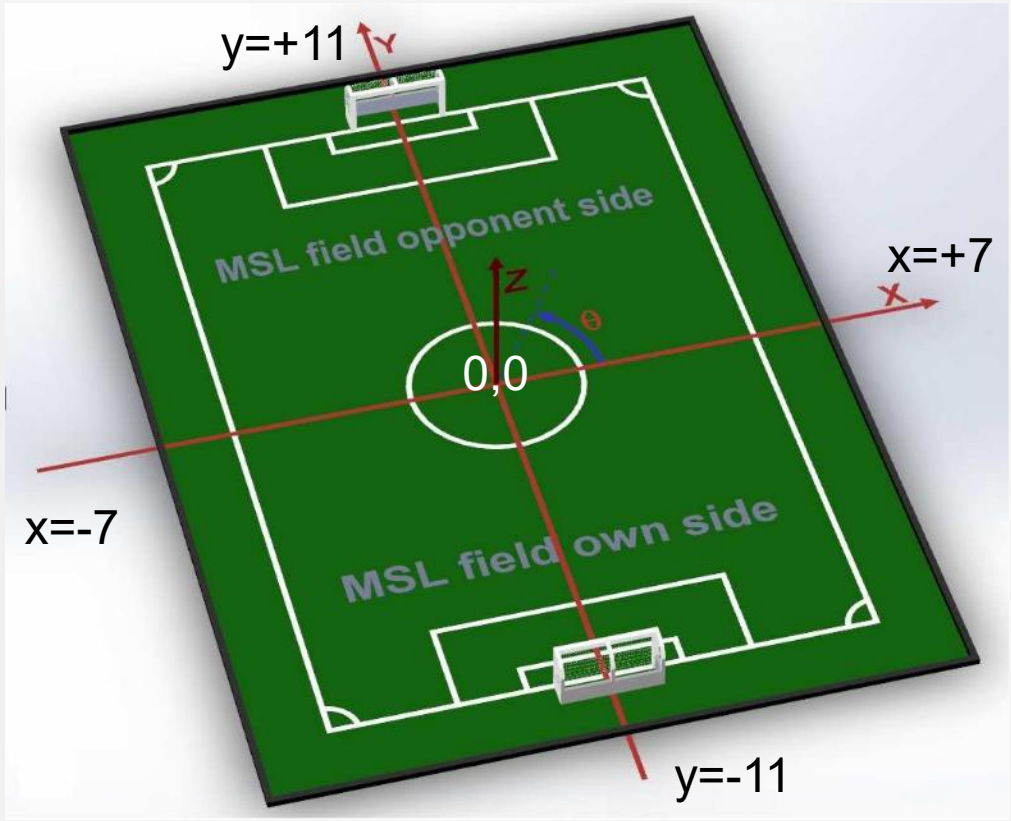
- x position
- y position
- θ angle of the front of the robot

Field position:

- the World coordinate system is defined as viewed from the own side of the field;
- (0,0) is the center of the field;
- the 'x' axis is positive to the right;
- the 'y' axis is positive towards the opponent goal;
- The 'θ' is 0-360 degrees with 90 towards enemy goal.

The Field seize :

- Length: minimum 18m, maximum 22m.
- Width: minimum 12m, maximum 14m.
- VDL field test field size is 18mx12m
- The 2022 tournament field size is 22m × 14m



Ground truth

Decawave technology to capture ground truth field position

- Decawave DWM1001 module
- Indoor positioning system
- Anchors/tags to capture position data



https://www.youtube.com/watch?v=hI8EaU5nOmI

Capture training data

- Field position
- in combination with Omni camara image, IMU/compass data, Zed2 image, etc

Precision Decawave

- Precision of 10 cm, but we sampled higher precisions in range of 5 cm



Gradient descent on the Free Energy

Minimize the Free energy by gradient descent on the Free Energy, example $\tilde{\mu}_{gx}$:

$$\dot{\tilde{\mu}}_{gx} = D\tilde{\mu}_{gx} - D^T\tilde{\Pi}_{gx}\tilde{\varepsilon}_{gx} + \left(\partial_{\tilde{\mu}_g}\tilde{f}(\tilde{\mu}_g)_x\right)^T\tilde{\Pi}_{gx}\tilde{\varepsilon}_{gx} + \sum_{landmarks}\left(\partial_{\tilde{\mu}_g}\tilde{g}(\tilde{\mu}_g)_x\right)^T\tilde{\Pi}_{px}\tilde{\varepsilon}_{px}$$

- $\left(\partial_{\tilde{\mu}_g}\tilde{f}(\tilde{\mu}_g)_x\right)^T = 0$ The motion is not dependent of the x position/speed/acceleration on the field location $\dot{\tilde{\mu}}_g$
  - Optionally take $f(\mu_g)_x = -\mu_{gx} + T_{pos_x}$ to drive to target position T_{pos_x} then $\left(\partial_{\mu_g} f(\mu_g)_x \cdot \mathrm{I}_{p+1}\right)^T$ to -1

$$\partial_{\tilde{\mu}_{gx}}\tilde{f}(\tilde{\mu}_{gx}) = \partial_{\tilde{\mu}_{gx}}\begin{bmatrix} f(\mu_{gx}) \\ \partial_{\mu_g}f(\mu_x)\cdot\mu'_x \\ \partial_{\mu_g}f(\mu_x)\cdot\mu''_x \\ \vdots \end{bmatrix} = \partial_{\mu_{gx}}f(\mu_{gx})\cdot \mathrm{I}_{p+1}$$

$$\begin{bmatrix}\tilde{\mu}_{gx}\\ \tilde{\mu}_{gy}\\ \tilde{\mu}_{g\theta}\end{bmatrix}_{t+\Delta t} = \begin{bmatrix}\tilde{\mu}_{gx}\\ \tilde{\mu}_{gy}\\ \tilde{\mu}_{g\theta}\end{bmatrix}_t + \Delta t \cdot \begin{bmatrix}\dot{\tilde{\mu}}_{gx}\\ \dot{\tilde{\mu}}_{gy}\\ \dot{\tilde{\mu}}_{g\theta}\end{bmatrix}_t$$

where:

$$\dot{\tilde{\mu}}_{gx} = \begin{bmatrix}\mu'_{gx}\\ \mu''_{gx}\\ 0\end{bmatrix} - \begin{bmatrix}0\\ \Pi_{Odo}(\mu'_{gx} - V'_x\ odometry)\\ \Pi_{IMU}(\mu''_{gx} - V''_x\ IMU)\end{bmatrix} - \sum_{landmarks}\begin{bmatrix}\Pi_{cam}\mathrm{mr_y J_{gx}}(g(\mu_g)_x - y_x)\\ 0\\ 0\end{bmatrix} - \begin{bmatrix}\Pi_{Odo}(\mu'_{gx} - V'_x\ odometry)\\ \Pi_{IMU}(\mu''_{gx} - V''_x\ IMU)\\ 0\end{bmatrix}$$